DHIRUBHAI AMBANI UNIVERSITY • FORMERLY DA-IICT

IT457 – Cloud Computing

BTech Final Year • Autumn Semester 2026

Week-1 Report

Freshworks on AWS:

(SaaS company built and scaled on cloud from India for global markets)

Project Theme: Startups & AWS Case Studies

Scaling a Global Multi-Tenant SaaS Platform from Single-Node RDS to Cell-Based Shells, Kubernetes, & Bedrock GenAI

Group-15 : Jayaditya Shah (202301254) • Hriday Asnani (202301282) • Maharshi Patel (202301486)

1. Sources Found

Primary sources were extracted from audited SEC Form 10-K/S-1 filings, official AWS case studies, and engineering publications authored by Freshworks platform architects and SREs.

#	Source (Type, Date)	What it gives us / Role in Case Study	Link
1	**Freshworks SEC Form 10-K** (Audited Filing; Feb 2024)	Audited FY21–FY23 revenue, gross margin (78.9% → 82.7%), $337.5M non-cancelable AWS purchase obligations, third-party hosting cost line items.	sec.gov
2	**Freshworks SEC Form 10-K** (Audited Filing; Feb 2025)	FY24 revenue ($720.4M, +20.8% YoY), gross margin (84.3%), hosting spend growth deceleration (+$1.6M vs +$8.5M FY22), $278.8M remaining AWS commitments.	sec.gov
3	**AWS Case Study: Freshworks** (Official Story; 2024)	Global multi-region expansion (London, Dubai, Frankfurt), 70,000+ customers across 140+ countries, Amazon Bedrock GenAI orchestration (CloudVerse).	aws.amazon.com
4	**B. Kumar: Scaling Web App** (Freshworks Eng; Jul 2020)	~500K peak RPM ingress, Passenger copy-on-write workers, Route 53 subdomain routing, Semian circuit breaking, automated MySQL `rds_kill` event.	freshworks.com
5	**Freshdesk Shell Architecture** (Freshworks Eng; 2021)	Cell-based HAProxy routing, ~60K RPM cap per isolated shell, Buffer Shell dynamic routing to quarantine degraded or runaway tenant shards.	freshworks.eng
6	**Managing Replication Lag** (Freshworks Eng; 2022)	GitHub Freno lag controller, 2-second replica threshold, throttling background Sidekiq workers to prevent secondary MySQL read starvation.	freshworks.eng
7	**Multi-Tenant Redis on EKS** (Freshworks Eng; 2023)	Custom Kubernetes Operator for Redis clusters, automated Route 53 DNS failover, algorithmic multi-AZ placement boosting fault tolerance by >50%.	freshworks.eng
8	**The Async Shift Analytics** (Freshworks Eng; 2024)	Refactoring 44M+ weekly queries from synchronous blocking APIs to asynchronous Kafka/Athena pipelines, scaling throughput 5× at $0 added compute cost.	freshworks.eng

Table 1: Primary authoritative and engineering sources collected for the Freshworks case study.

2. Case Study Salient Points

Background & Scale: Founded in Chennai, India in 2010 (NASDAQ: `FRSH`), Freshworks built natively on AWS to bypass physical data center capital costs. Today, it powers customer engagement and ITSM across 70,000+ businesses in 140+ countries. Ingress traffic peaks at ~500,000 Requests Per Minute (RPM) across its monolithic Ruby on Rails web core.

The Core Bottleneck: Rapid international customer onboarding outpaced early monolithic infrastructure. Monolithic Rails worker pools saturated during traffic bursts, while a single primary MySQL database suffered row-lock contention and connection exhaustion, risking multi-tenant cascade failures.

Three-Stage Architectural Evolution:

- **Stage 1 (2010–2014) — Startup Monolith:** Single monolithic Rails app on Amazon EC2 backed by Amazon RDS for MySQL 5.1. Suffered from unindexed query locks, disk IOPS saturation, and lack of tenant isolation.
- **Stage 2 (2015–2019) — Sharded Multi-AZ Scale:** Horizontal functional sharding across multiple RDS Aurora clusters, read replicas, ElastiCache tiers, and EC2 Auto Scaling Groups behind Application Load Balancers.
- **Stage 3 (2020–Present) — Cell-Based & Cloud-Native:** Layer-7 "Shell Architecture" on HAProxy (~60K RPM cap), self-hosted Redis clusters on Amazon EKS via custom operators, automated MySQL lock killers, Freno-controlled Sidekiq queues, and Bedrock GenAI.

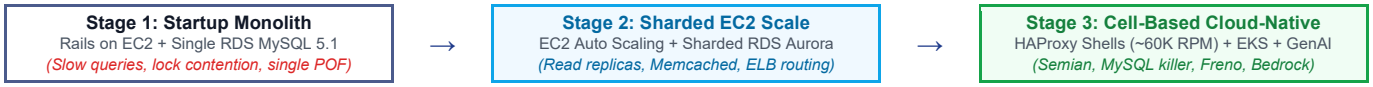


Figure 1: Evolution of Freshworks' AWS setup across three architectural stages.

3. Architectural Outline

Freshworks' production architecture decouples user-facing request handling, stateful caching, asynchronous worker processing, and analytical execution across specialized cloud tiers:

- **Request Ingress & Shell Routing:** Multi-tenant subdomains resolve via Amazon Route 53 (e.g., `tenant.freshdesk.com`) to AWS Application Load Balancers. An internal HAProxy routing tier implements *Shell Architecture*, mapping tenants to isolated compute/DB clusters capped at ~60,000 RPM. Degraded or runaway tenants are dynamically diverted to a dedicated *Buffer Shell*, isolating blast radiuses.
- **Compute & In-Process Resiliency:** Nginx reverse proxies route requests to Phusion Passenger worker pools running Rails. Local Envoy sidecars manage service mesh egress. Critical database and Redis calls are wrapped in Shopify's **Semian** circuit breakers, tripping within 5 consecutive failures and failing fast in <2ms to prevent worker pool starvation during Multi-AZ failovers.
- **Database Layer & Transaction Watchdog:** Customer data resides on horizontally partitioned Amazon Aurora MySQL clusters (Primary + Read Replicas). An automated in-database MySQL event (`kill_long_transactions`) inspects `INNODB_TRX` every 5 seconds and forcibly terminates queries running >25s via `CALL mysql.rds_kill()`, freeing row locks and connection slots.
- **Stateful Caching on EKS:** Migrated from unmanaged EC2 instances to self-hosted Redis clusters on Amazon EKS. A custom Kubernetes Operator automates Route 53 DNS failovers and executes an algorithmic multi-AZ placement engine, improving cluster fault tolerance by >50%.
- **Asynchronous Job Processing:** Non-blocking operations queue into Sidekiq backed by Redis. To protect database secondaries, workers query GitHub **Freno** before execution; if replication lag exceeds 2 seconds, Freno postpones job execution by 60 seconds.
- **The Async Analytics Shift:** Analytical queries (>44M/week) were refactored from synchronous web threads into an event-driven Kafka pipeline. Jobs execute asynchronously against Amazon Athena and S3 data lakes, boosting query capacity 5× at $0 added compute spend.
- **Observability Tier:** Centralized logs flow to Amazon S3 via Fluentbit, with Datadog APM, Prometheus/Grafana, and Sentry for real-time error tracking.

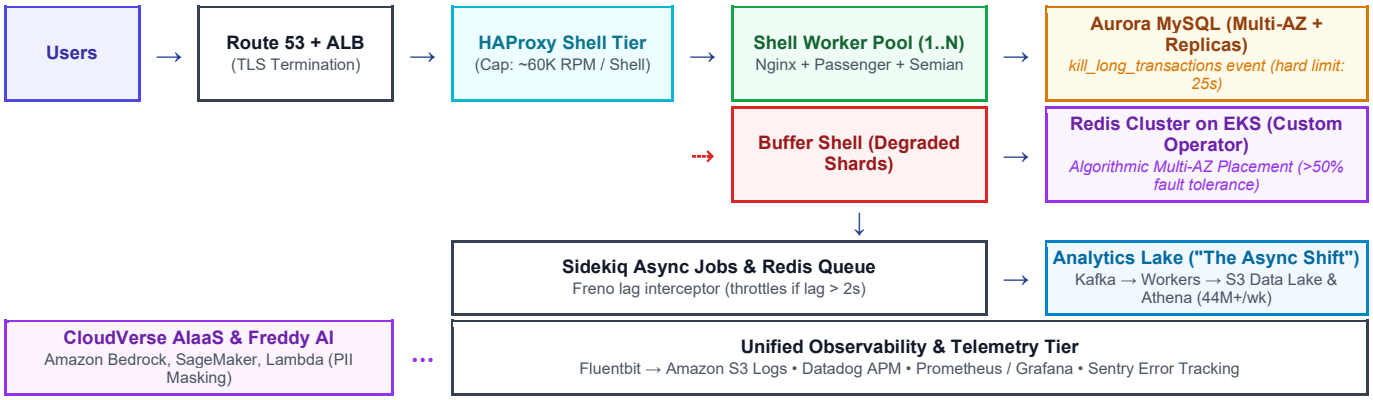


Figure 2: Production architecture of Freshworks on AWS as mapped from verified primary sources.

Layer	AWS Services	Role in Freshworks Platform
Ingress / Edge	Route 53, ALB, HAProxy	Subdomain resolution, TLS termination, layer-7 Shell routing (~60K RPM cap), Buffer Shell quarantine.
Compute / App	Amazon EC2, EKS, Envoy	Passenger Rails multi-tenant app servers, local Envoy sidecars, Semian fail-fast circuit breakers.
OLTP Database	Amazon Aurora MySQL	Horizontally sharded multi-AZ clusters with scheduled `kill_long_transactions` watchdog (>25s).
Stateful Cache	Amazon EKS, Route 53	Self-hosted Redis managed by custom K8s Operator; algorithmic multi-AZ placement (>50% resilience).
Async Jobs	Sidekiq, Redis, Freno	Background job fleet throttled by GitHub Freno when database replica lag exceeds 2 seconds.
Analytics Lake	Kafka, S3, Athena, Glue	"The Async Shift": 44M+ weekly queries decoupled from synchronous APIs to background execution.
Generative AI	Amazon Bedrock, SageMaker	CloudVerse AIaaS platform orchestrating Freddy AI models with token rate limits and PII sanitization.

Table 2: Freshworks cloud architecture components extracted from authoritative technical sources.

4. Cost and Outcome Data

Audited financial disclosures in Freshworks' SEC filings and technical benchmarks demonstrate how proactive cloud engineering decoupled corporate revenue growth from cloud hosting infrastructure expenditures:

Metric	Before / Baseline	After / Reported Outcome	Source
Annual Subscription Revenue	$371.0M (FY2021)	**$720.4M** (FY2024; +94.2% over 3 years)	1, 2
GAAP Gross Margin	78.95% (FY2021)	**84.27%** (FY2024; +532 bps expansion)	1, 2
YoY Hosting Cost Increase Deceleration	+$8.5M (FY2022)	**+$1.6M** (FY2024; -81.2% deceleration)	1, 2
Hosting Spend as % of New Revenue	6.69% of new revenue (FY22)	**1.29%** of new revenue (FY24)	1, 2
Non-Cancelable AWS Commitments	Not publicly reported	**$337.5M** (FY23) → **$278.8M** (FY24)	1, 2
Peak Ingress Request Throughput	Startup volume	**~500,000 RPM** on monolithic core	4, 5
Shell Architecture Cell Capacity Cap	Unpartitioned monolith	Capped at **~60,000 RPM** per shell	5
MySQL Transaction Lock Duration Limit	Unbounded (hangs workers)	Hard termination at **25 seconds**	4
Stateful Redis Fault Tolerance on EKS	Unmanaged standalone nodes	**>50% increase** in fault resilience	7
Background Queue Replica Lag Limit	Unchecked replica lag	Throttled if MySQL lag **> 2 seconds**	6
Analytics Query Throughput (Async Shift)	Synchronous blocking model	**44M+ queries/week** (5× load at $0 extra cost)	8

Table 3: Financial, capacity, and operational outcome metrics extracted from audited SEC filings and technical sources.

Core Resilience Mechanisms: Solution Highlights

- **Blast Radius Containment via Shell Architecture:** Freshworks partitions multi-tenant infrastructure into independent layer-7 "Shells" capped at ~60,000 RPM. A dynamic *Buffer Shell* absorbs noisy or degraded tenants without cascading connection exhaustion or database locks across healthy tenant clusters.
- **Fail-Fast In-Process Circuit Breaking (Semian):** Wrapping database and cache drivers in Shopify's Semian ensures that when database locks or Multi-AZ failovers occur, workers fail within <2ms after 5 successive timeouts. This preserves Rails worker availability for cached endpoints, avoiding complete thread pool starvation.
- **Proactive In-Database Lock Watchdogs:** The `kill_long_transactions` event continuously monitors InnoDB locks and terminates runaway queries exceeding 25 seconds via `CALL mysql.rds_kill()`. This guarantees that unindexed queries from one tenant cannot permanently exhaust database connection pools.
- **Algorithmic Multi-AZ Stateful Placement:** Rather than paying high managed costs for ElastiCache, Freshworks deployed self-hosted Redis on Amazon EKS. Its custom operator algorithmically ensures that replica pods never co-locate in the same Availability Zone as primary nodes, improving fault tolerance by >50%.
- **Adaptive Replica Lag Throttling (GitHub Freno):** Sidekiq workers dynamically poll Freno prior to executing write-intensive jobs. When MySQL replication lag exceeds 2 seconds, jobs are delayed by 60 seconds, eliminating secondary read starvation and replica desynchronization.
- **Decoupled Asynchronous Analytics ("The Async Shift"):** Shifting 44M+ weekly analytics queries from synchronous HTTP request threads to asynchronous Kafka/Athena pipelines expanded query throughput by 5× while incurring $0 in additional data warehouse compute costs.

Cloud Unit Economics & Outcome Synthesis

Freshworks' trajectory illustrates the transition from brute-force cloud adoption to refined cloud unit economics. In early scaling stages, third-party hosting costs expanded rapidly (consuming 6.69% of all incremental revenue in FY22). However, between FY22 and FY24, hosting cost growth decelerated from +$8.5M to +$1.6M annually, representing just 1.29% of new revenue. Over this same period, corporate GAAP gross margins expanded from 78.9% to 84.3%, while revenue grew 94.2% to $720.4M.

These gains were not achieved by over-provisioning raw EC2 compute, but by rigorous architectural discipline: isolating blast radiuses at layer 7, failing fast with circuit breakers, terminating stalled locks proactively, and decoupling blocking analytics. SEC Note 8 disclosures confirm $337.5M in non-cancelable purchase commitments through 2028 (predominantly AWS), demonstrating that hyper-scale SaaS profitability depends on algorithmic architectural optimization rather than raw cloud capacity expansion.

Open Data Boundaries: In compliance with academic rigor, proprietary trade secrets—such as itemized AWS monthly bills (exact dollar split across EC2, Aurora, and Bedrock tokens) and internal cost-per-ticket metrics—remain confidential to Freshworks and are recognized as data boundaries.